

Data Mining and Machine Learning

A Master Course

Master program on Data Science

Tran Trong Hieu¹

¹VNU - University of Engineering and Technology

Chapter 21: Support Vector Machines

Hyperplanes

Let $\mathbf{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ be a classification dataset, with n points in a d -dimensional space. We assume that there are only two class labels, that is, $y_i \in \{+1, -1\}$, denoting the positive and negative classes.

A hyperplane in d dimensions is given as the set of all points $\mathbf{x} \in \mathbb{R}^d$ that satisfy the equation $h(\mathbf{x}) = 0$, where $h(\mathbf{x})$ is the *hyperplane function*:

$$h(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b$$

Here, $\mathbf{w}$ is a d dimensional *weight vector* and b is a scalar, called the *bias*.

For points that lie on the hyperplane, we have

$$h(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = 0$$

The weight vector $\mathbf{w}$ specifies the direction that is orthogonal or normal to the hyperplane, which fixes the orientation of the hyperplane, whereas the bias b fixes the offset of the hyperplane in the d -dimensional space, i.e., where the hyperplane intersects each of the axes:

$$w_i x_i = -b \quad \text{or} \quad x_i = \frac{-b}{w_i}$$

Separating Hyperplane

A hyperplane splits the d -dimensional data space into two *half-spaces*.

A dataset is said to be *linearly separable* if each half-space has points only from a single class.

If the input dataset is linearly separable, then we can find a *separating* hyperplane $h(\mathbf{x}) = 0$, such that for all points labeled $y_i = -1$, we have $h(\mathbf{x}_i) < 0$, and for all points labeled $y_i = +1$, we have $h(\mathbf{x}_i) > 0$.

The hyperplane function $h(\mathbf{x})$ thus serves as a linear classifier or a linear discriminant, which predicts the class y for any given point $\mathbf{x}$, according to the decision rule:

$$y = \begin{cases} +1 & \text{if } h(\mathbf{x}) > 0 \\ -1 & \text{if } h(\mathbf{x}) < 0 \end{cases}$$

Geometry of a Hyperplane: Distance

Consider a point $\mathbf{x} \in \mathbb{R}^d$ that does not lie on the hyperplane. Let $\mathbf{x}_p$ be the orthogonal projection of $\mathbf{x}$ on the hyperplane, and let $\mathbf{r} = \mathbf{x} - \mathbf{x}_p$. Then we can write $\mathbf{x}$ as

$$\mathbf{x} = \mathbf{x}_p + \mathbf{r} = \mathbf{x}_p + r \frac{\mathbf{w}}{\|\mathbf{w}\|}$$

where r is the *directed distance* of the point $\mathbf{x}$ from $\mathbf{x}_p$.

To obtain an expression for r , consider the value $h(\mathbf{x})$, we have:

$$h(\mathbf{x}) = h\left(\mathbf{x}_p + r \frac{\mathbf{w}}{\|\mathbf{w}\|}\right) = \mathbf{w}^T \left(\mathbf{x}_p + r \frac{\mathbf{w}}{\|\mathbf{w}\|}\right) + b = r \|\mathbf{w}\|$$

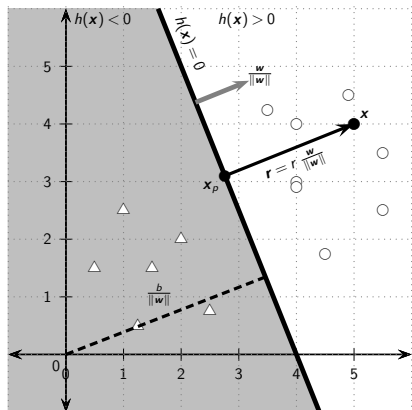
The directed distance r of point $\mathbf{x}$ to the hyperplane is thus:

$$r = \frac{h(\mathbf{x})}{\|\mathbf{w}\|}$$

To obtain distance, which must be non-negative, we multiply r by the class label y_i of the point $\mathbf{x}_i$ because when $h(\mathbf{x}_i) < 0$, the class is -1 , and when $h(\mathbf{x}_i) > 0$ the class is $+1$:

$$\delta_i = \frac{y_i h(\mathbf{x}_i)}{\|\mathbf{w}\|}$$

Geometry of a Hyperplane in 2D



$$\mathbf{p} = (p_1, p_2) = (4, 0), \quad \mathbf{q} = (q_1, q_2) = (2, 5)$$

$$-\frac{w_1}{w_2} = \frac{q_2 - p_2}{q_1 - p_1} = \frac{5 - 0}{2 - 4} = -\frac{5}{2}$$

Given $(4, 0)$, the offset b is:

$$b = -5x_1 - 2x_2 = -5 \cdot 4 - 2 \cdot 0 = -20$$

Given $\mathbf{w} = \begin{pmatrix} 5 \\ 2 \end{pmatrix}$ and $b = -20$:

$$h(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = \begin{pmatrix} 5 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} - 20 = 0$$

$$\delta = y r = -1 r = \frac{-b}{\|\mathbf{w}\|} = \frac{-(-20)}{\sqrt{29}} = 3.71$$

Margin and Support Vectors

The distance of a point $\mathbf{x}$ from the hyperplane $h(\mathbf{x}) = 0$ is thus given as

$$\delta = y r = \frac{y h(\mathbf{x})}{\|\mathbf{w}\|}$$

The *margin* is the minimum distance of a point from the separating hyperplane:

$$\delta^* = \min_{\mathbf{x}_i} \left\{ \frac{y_i (\mathbf{w}^T \mathbf{x}_i + b)}{\|\mathbf{w}\|} \right\}$$

All the points (or vectors) that achieve the minimum distance are called *support vectors* for the hyperplane. They satisfy the condition:

$$\delta^* = \frac{y^* (\mathbf{w}^T \mathbf{x}^* + b)}{\|\mathbf{w}\|}$$

where y^* is the class label for $\mathbf{x}^*$.

Canonical Hyperplane

Multiplying the hyperplane equation on both sides by some scalar s yields an equivalent hyperplane:

$$s h(\mathbf{x}) = s \mathbf{w}^T \mathbf{x} + s b = (s\mathbf{w})^T \mathbf{x} + (sb) = 0$$

To obtain the unique or *canonical* hyperplane, we choose the scalar $s = \frac{1}{y^*(\mathbf{w}^T \mathbf{x}^* + b)}$ so that the absolute distance of a support vector from the hyperplane is 1, i.e., the margin is

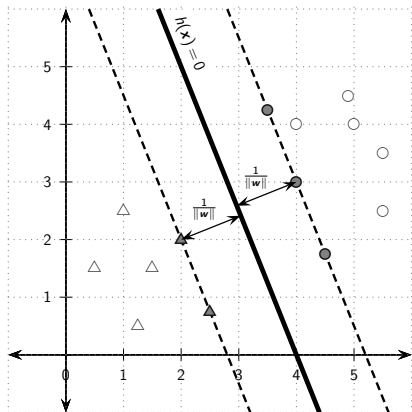
$$\delta^* = \frac{y^*(\mathbf{w}^T \mathbf{x}^* + b)}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}$$

For the canonical hyperplane, for each support vector $\mathbf{x}_i^*$ (with label y_i^*), we have $y_i^* h(\mathbf{x}_i^*) = 1$, and for any point that is not a support vector we have $y_i h(\mathbf{x}_i) > 1$. Over all points, we have

$$y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \text{ for all points } \mathbf{x}_i \in D$$

Separating Hyperplane: Margin and Support Vectors

Shaded points are support vectors



$$h(x) = \begin{pmatrix} 5 \\ 2 \end{pmatrix}^T x - 20 = 0$$

Given $x^* = (2, 2)^T$, $y^* = -1$.

$$s = \frac{1}{y^* h(x^*)} = \frac{1}{-1 \left(\begin{pmatrix} 5 \\ 2 \end{pmatrix}^T \begin{pmatrix} 2 \\ 2 \end{pmatrix} - 20 \right)} = \frac{1}{6}$$

$$w = \frac{1}{6} \begin{pmatrix} 5 \\ 2 \end{pmatrix} = \begin{pmatrix} 5/6 \\ 2/6 \end{pmatrix} \quad b = \frac{-20}{6}$$

$$h(x) = \begin{pmatrix} 5/6 \\ 2/6 \end{pmatrix}^T x - 20/6 = \begin{pmatrix} 0.833 \\ 0.333 \end{pmatrix}^T x - 3.33$$

$$\delta^* = \frac{y^* h(x^*)}{\|w\|} = \frac{1}{\sqrt{\left(\frac{5}{6}\right)^2 + \left(\frac{2}{6}\right)^2}} = \frac{6}{\sqrt{29}} = 1.114$$

SVM: Linear and Separable Case

Assume that the points are linearly separable, that is, there exists a separating hyperplane that perfectly classifies each point.

The goal of SVMs is to choose the canonical hyperplane, h^* , that yields the maximum margin among all possible separating hyperplanes

$$h^* = \arg \max_{\mathbf{w}, b} \left\{ \frac{1}{\|\mathbf{w}\|} \right\}$$

We can obtain an equivalent minimization formulation:

Objective Function: $\min_{\mathbf{w}, b} \left\{ \frac{\|\mathbf{w}\|^2}{2} \right\}$

Linear Constraints: $y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \forall \mathbf{x}_i \in D$

SVM: Linear and Separable Case

We turn the constrained SVM optimization into an unconstrained one by introducing a Lagrange multiplier α_i for each constraint. The new objective function, called the *Lagrangian*, then becomes

$$\min L = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i \left(y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 \right)$$

L should be minimized w.r.t. $\mathbf{w}$ and b , and it should be maximized w.r.t. α_i . Taking the derivative of L with respect to $\mathbf{w}$ and b , and setting those to zero, we obtain

$$\frac{\partial}{\partial \mathbf{w}} L = \mathbf{w} - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i = 0 \quad \text{or} \quad \mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

$$\frac{\partial}{\partial b} L = \sum_{i=1}^n \alpha_i y_i = 0$$

We can see that $\mathbf{w}$ can be expressed as a linear combination of the data points $\mathbf{x}_i$, with the signed Lagrange multipliers, $\alpha_i y_i$, serving as the coefficients.

Further, the sum of the signed Lagrange multipliers, $\alpha_i y_i$, must be zero.

SVM: Linear and Separable Case

Incorporating $\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$ and $\sum_{i=1}^n \alpha_i y_i = 0$ into the Lagrangian we obtain the new *dual Lagrangian* objective function, which is specified purely in terms of the Lagrange multipliers:

Objective Function:
$$\max_{\alpha} L_{dual} = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

Linear Constraints: $\alpha_i \geq 0, \forall i \in \mathbf{D}, \text{ and } \sum_{i=1}^n \alpha_i y_i = 0$

where $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_n)^T$ is the vector comprising the Lagrange multipliers.

L_{dual} is a convex quadratic programming problem (note the $\alpha_i \alpha_j$ terms), which admits a unique optimal solution.

SVM: Linear and Separable Case

Once we have obtained the α_i values for $i = 1, \dots, n$, we can solve for the weight vector $\mathbf{w}$ and the bias b . Each of the Lagrange multipliers α_i satisfies the KKT conditions at the optimal solution:

$$\alpha_i (y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1) = 0$$

which gives rise to two cases:

- 1) $\alpha_i = 0$, or
- 2) $y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 = 0$, which implies $y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$

This is a very important result because if $\alpha_i > 0$, then $y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$, and thus the point $\mathbf{x}_i$ must be a support vector.

On the other hand, if $y_i(\mathbf{w}^T \mathbf{x}_i + b) > 1$, then $\alpha_i = 0$, that is, if a point is not a support vector, then $\alpha_i = 0$.

Linear and Separable Case: Weight Vector and Bias

Once we know α_i for all points, we can compute the weight vector $\mathbf{w}$ by taking the summation only for the support vectors:

$$\mathbf{w} = \sum_{i, \alpha_i > 0} \alpha_i y_i \mathbf{x}_i$$

Only the support vectors determine $\mathbf{w}$, since $\alpha_i = 0$ for other points. To compute the bias b , we first compute one solution b_i , per support vector, as follows:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1, \text{ which implies } b_i = \frac{1}{y_i} - \mathbf{w}^T \mathbf{x}_i = y_i - \mathbf{w}^T \mathbf{x}_i$$

The bias b is taken as the average value:

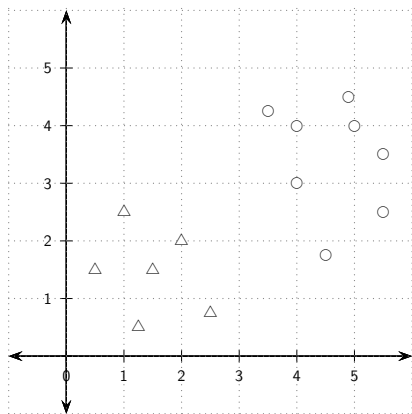
$$b = \text{avg}_{\alpha_i > 0} \{b_i\}$$

Given the optimal hyperplane function $h(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$, for any new point $\mathbf{z}$, we predict its class as

$$\hat{y} = \text{sign}(h(\mathbf{z})) = \text{sign}(\mathbf{w}^T \mathbf{z} + b)$$

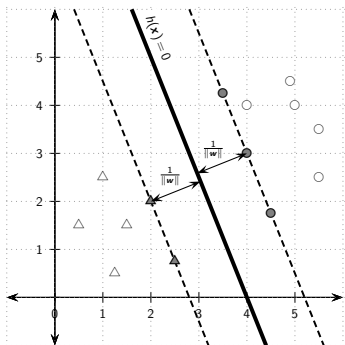
where the $\text{sign}(\cdot)$ function returns $+1$ if its argument is positive, and -1 if its argument is negative.

Example Dataset: Separable Case



x_i	x_{i1}	x_{i2}	y_i
x_1	3.5	4.25	+1
x_2	4	3	+1
x_3	4	4	+1
x_4	4.5	1.75	+1
x_5	4.9	4.5	+1
x_6	5	4	+1
x_7	5.5	2.5	+1
x_8	5.5	3.5	+1
x_9	0.5	1.5	-1
x_{10}	1	2.5	-1
x_{11}	1.25	0.5	-1
x_{12}	1.5	1.5	-1
x_{13}	2	2	-1
x_{14}	2.5	0.75	-1

Optimal Separating Hyperplane



Solving the L_{dual} quadratic program yields

$\mathbf{x}_i$	x_{i1}	x_{i2}	y_i	α_i
$\mathbf{x}_1$	3.5	4.25	+1	0.0437
$\mathbf{x}_2$	4	3	+1	0.2162
$\mathbf{x}_4$	4.5	1.75	+1	0.1427
$\mathbf{x}_{13}$	2	2	-1	0.3589
$\mathbf{x}_{14}$	2.5	0.75	-1	0.0437

The weight vector and bias are:

$$\mathbf{w} = \sum_{i, \alpha_i > 0} \alpha_i y_i \mathbf{x}_i = \begin{pmatrix} 0.833 \\ 0.334 \end{pmatrix}$$

$$b = \text{avg}\{b_i\} = -3.332$$

The optimal hyperplane is given as follows:

$$h(\mathbf{x}) = \begin{pmatrix} 0.833 \\ 0.334 \end{pmatrix}^T \mathbf{x} - 3.332 = 0$$

Soft Margin SVM: Linear and Nonseparable Case

The assumption that the dataset be perfectly linearly separable is unrealistic. SVMs can handle non-separable points by introducing *slack variables* ξ_i as follows:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$$

where $\xi_i \geq 0$ is the slack variable for point $\mathbf{x}_i$, which indicates how much the point violates the separability condition, that is, the point may no longer be at least $1/\|\mathbf{w}\|$ away from the hyperplane.

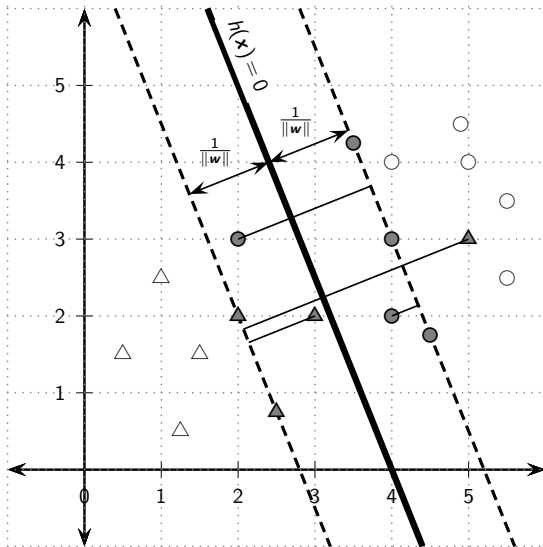
The slack values indicate three types of points. If $\xi_i = 0$, then the corresponding point $\mathbf{x}_i$ is at least $\frac{1}{\|\mathbf{w}\|}$ away from the hyperplane.

If $0 < \xi_i < 1$, then the point is within the margin and still correctly classified, that is, it is on the correct side of the hyperplane.

However, if $\xi_i \geq 1$ then the point is misclassified and appears on the wrong side of the hyperplane.

Soft Margin Hyperplane

Shaded points are the support vectors



SVM: Soft Margin or Linearly Non-separable Case

In the nonseparable case, also called the *soft margin* the SVM objective function is

$$\begin{aligned} \textbf{Objective Function: } & \min_{\mathbf{w}, b, \xi_i} \left\{ \frac{\|\mathbf{w}\|^2}{2} + C \sum_{i=1}^n (\xi_i)^k \right\} \\ \textbf{Linear Constraints: } & y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \forall \mathbf{x}_i \in \mathbf{D} \\ & \xi_i \geq 0 \quad \forall \mathbf{x}_i \in \mathbf{D} \end{aligned}$$

where C and k are constants that incorporate the cost of misclassification.

The term $\sum_{i=1}^n (\xi_i)^k$ gives the *loss*, that is, an estimate of the deviation from the separable case.

The scalar C is a *regularization constant* that controls the trade-off between maximizing the margin or minimizing the loss. For example, if $C \rightarrow 0$, then the loss component essentially disappears, and the objective defaults to maximizing the margin. On the other hand, if $C \rightarrow \infty$, then the margin ceases to have much effect, and the objective function tries to minimize the loss.

SVM: Soft Margin Loss Function

The constant k governs the form of the loss. When $k = 1$, called *hinge loss*, the goal is to minimize the sum of the slack variables, whereas when $k = 2$, called *quadratic loss*, the goal is to minimize the sum of the squared slack variables.

Hinge Loss: Assuming $k = 1$, the SVM dual Lagrangian is given as

$$\max_{\alpha} L_{dual} = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

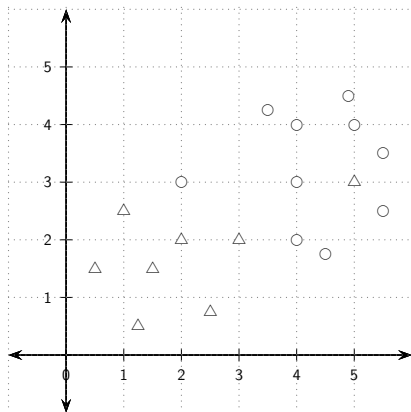
The only difference from the separable case is that $0 \leq \alpha_i \leq C$.

Quadratic Loss: Assuming $k = 2$, the dual objective is:

$$\max_{\alpha} L_{dual} = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \left(\mathbf{x}_i^T \mathbf{x}_j + \frac{1}{2C} \delta_{ij} \right)$$

where δ is the *Kronecker delta* function, defined as $\delta_{ij} = 1$ if and only if $i = j$.

Example Dataset: Linearly Non-separable Case



x_i	x_{i1}	x_{i2}	y_i
x_1	3.5	4.25	+1
x_2	4	3	+1
x_3	4	4	+1
x_4	4.5	1.75	+1
x_5	4.9	4.5	+1
x_6	5	4	+1
x_7	5.5	2.5	+1
x_8	5.5	3.5	+1
x_9	0.5	1.5	-1
x_{10}	1	2.5	-1
x_{11}	1.25	0.5	-1
x_{12}	1.5	1.5	-1
x_{13}	2	2	-1
x_{14}	2.5	0.75	-1
x_{15}	4	2	+1
x_{16}	2	3	+1
x_{17}	3	2	-1
x_{18}	5	3	-1

Example Dataset: Linearly Non-separable Case

Let $k = 1$ and $C = 1$, then solving the L_{dual} yields the following support vectors and Lagrangian values α_i :

$\mathbf{x}_i$	x_{i1}	x_{i2}	y_i	α_i
$\mathbf{x}_1$	3.5	4.25	+1	0.0271
$\mathbf{x}_2$	4	3	+1	0.2162
$\mathbf{x}_4$	4.5	1.75	+1	0.9928
$\mathbf{x}_{13}$	2	2	-1	0.9928
$\mathbf{x}_{14}$	2.5	0.75	-1	0.2434
$\mathbf{x}_{15}$	4	2	+1	1
$\mathbf{x}_{16}$	2	3	+1	1
$\mathbf{x}_{17}$	3	2	-1	1
$\mathbf{x}_{18}$	5	3	-1	1

The optimal hyperplane is given as follows:

$$h(\mathbf{x}) = \begin{pmatrix} 0.834 \\ 0.333 \end{pmatrix}^T \mathbf{x} - 3.334 = 0$$

Example Dataset: Linearly Non-separable Case

The slack $\xi_i = 0$ for all points that are not support vectors, and also for those support vectors that are on the margin. Slack is positive only for the remaining support vectors and it can be computed as: $\xi_i = 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)$.

Thus, for all support vectors not on the margin, we have

$\mathbf{x}_i$	$\mathbf{w}^T \mathbf{x}_i$	$\mathbf{w}^T \mathbf{x}_i + b$	$\xi_i = 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)$
$\mathbf{x}_{15}$	4.001	0.667	0.333
$\mathbf{x}_{16}$	2.667	-0.667	1.667
$\mathbf{x}_{17}$	3.167	-0.167	0.833
$\mathbf{x}_{18}$	5.168	1.834	2.834

The total slack is given as

$$\sum_i \xi_i = \xi_{15} + \xi_{16} + \xi_{17} + \xi_{18} = 0.333 + 1.667 + 0.833 + 2.834 = 5.667$$

The slack variable $\xi_i > 1$ for those points that are misclassified (i.e., are on the wrong side of the hyperplane), namely $\mathbf{x}_{16} = (3, 3)^T$ and $\mathbf{x}_{18} = (5, 3)^T$. The other two points are correctly classified, but lie within the margin, and thus satisfy $0 < \xi_i < 1$.

Kernel SVM: Nonlinear Case

The linear SVM approach can be used for datasets with a nonlinear decision boundary via the kernel trick.

Conceptually, the idea is to map the original d -dimensional points $\mathbf{x}_i$ in the input space to points $\phi(\mathbf{x}_i)$ in a high-dimensional feature space via some nonlinear transformation ϕ .

Given the extra flexibility, it is more likely that the points $\phi(\mathbf{x}_i)$ might be linearly separable in the feature space.

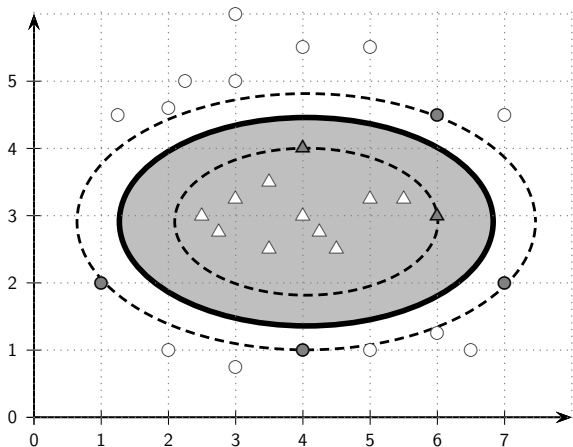
A linear decision surface in feature space actually corresponds to a nonlinear decision surface in the input space.

Further, the kernel trick allows us to carry out all operations via the kernel function computed in input space, rather than having to map the points into feature space.

Nonlinear SVM

There is no linear classifier that can discriminate between the points. However, there exists a perfect quadratic classifier that can separate the two classes.

$$\phi(\mathbf{x}) = (\sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2)^T$$



Nonlinear SVMs: Kernel Trick

To apply the kernel trick for nonlinear SVM classification, we have to show that all operations require only the kernel function:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

Applying ϕ to each point, we can obtain the new dataset in the feature space $\mathbf{D}_\phi = \{\phi(\mathbf{x}_i), y_i\}_{i=1}^n$.

The SVM objective function in feature space is given as

Objective Function: $\min_{\mathbf{w}, b, \xi_i} \left\{ \frac{\|\mathbf{w}\|^2}{2} + C \sum_{i=1}^n (\xi_i)^k \right\}$

Linear Constraints: $y_i (\mathbf{w}^T \phi(\mathbf{x}_i) + b) \geq 1 - \xi_i$, and $\xi_i \geq 0$, $\forall \mathbf{x}_i \in \mathbf{D}$

where $\mathbf{w}$ is the weight vector, b is the bias, and ξ_i are the slack variables, all in feature space.

Nonlinear SVMs: Kernel Trick

For hinge loss, the dual Lagrangian in feature space is given as

$$\begin{aligned}\max_{\alpha} L_{dual} &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) \\ &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)\end{aligned}$$

Subject to the constraints that $0 \leq \alpha_i \leq C$, and $\sum_{i=1}^n \alpha_i y_i = 0$.

The dual Lagrangian depends only on the dot product between two vectors in feature space $\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) = K(\mathbf{x}_i, \mathbf{x}_j)$, and thus we can solve the optimization problem using the kernel matrix $\mathbf{K} = \{K(\mathbf{x}_i, \mathbf{x}_j)\}_{i,j=1,\dots,n}$.

For quadratic loss, the dual Lagrangian corresponds to the use of a new kernel

$$K_q(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j + \frac{1}{2C} \delta_{ij} = K(\mathbf{x}_i, \mathbf{x}_j) + \frac{1}{2C} \delta_{ij}$$

Nonlinear SVMs: Weight Vector and Bias

We cannot directly obtain the weight vector without transforming the points, since

$$\mathbf{w} = \sum_{\alpha_i > 0} \alpha_i y_i \phi(\mathbf{x}_i)$$

However, we can compute the bias via kernel operations, since

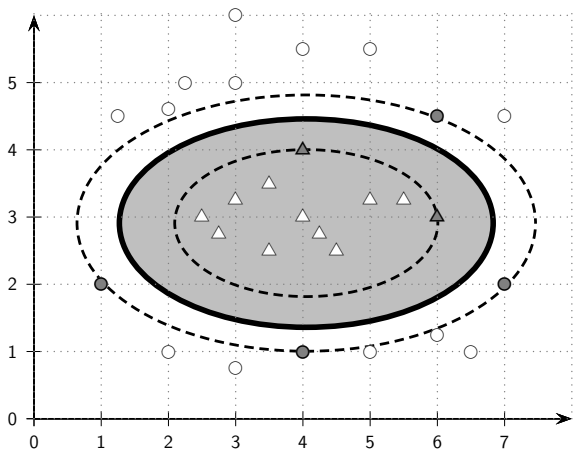
$$b_i = y_i - \mathbf{w}^T \phi(\mathbf{x}_i) = y_i - \sum_{\alpha_j > 0} \alpha_j y_j K(\mathbf{x}_j, \mathbf{x}_i)$$

Likewise, we can predict the class for a new point $\mathbf{z}$ as follows:

$$\hat{y} = \text{sign}(\mathbf{w}^T \phi(\mathbf{z}) + b) = \text{sign} \left(\sum_{\alpha_i > 0} \alpha_i y_i K(\mathbf{x}_i, \mathbf{z}) + b \right)$$

All SVM operations can be carried out in terms of the kernel function $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$. Thus, any nonlinear kernel function can be used to do nonlinear classification in the input space.

Nonlinear SVM: Inhomogeneous Quadratic Kernel



The optimal quadratic hyperplane is obtained by setting $C = 4$, and using an inhomogeneous polynomial kernel of degree $q = 2$:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^2$$

Nonlinear SVM: Inhomogeneous Quadratic Kernel

ϕ maps $\mathbf{x}_i$ into feature space as follows:

$$\phi(\mathbf{x} = (x_1, x_2)^T) = \left(1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2\right)^T$$

$\mathbf{x}_1 = (1, 2)^T$ is transformed into

$$\phi(\mathbf{x}_i) = \left(1, \sqrt{2} \cdot 1, \sqrt{2} \cdot 2, 1^2, 2^2, \sqrt{2} \cdot 1 \cdot 2\right)^T = (1, 1.41, 2.83, 1, 2, 2.83)^T$$

Solving L_{dual} , we found the following six support vectors:

$\mathbf{x}_i$	$(x_{i1}, x_{i2})^T$	$\phi(\mathbf{x}_i)$	y_i	α_i
$\mathbf{x}_1$	$(1, 2)^T$	$(1, 1.41, 2.83, 1, 4, 2.83)^T$	+1	0.6198
$\mathbf{x}_2$	$(4, 1)^T$	$(1, 5.66, 1.41, 16, 1, 5.66)^T$	+1	2.069
$\mathbf{x}_3$	$(6, 4.5)^T$	$(1, 8.49, 6.36, 36, 20.25, 38.18)^T$	+1	3.803
$\mathbf{x}_4$	$(7, 2)^T$	$(1, 9.90, 2.83, 49, 4, 19.80)^T$	+1	0.3182
$\mathbf{x}_5$	$(4, 4)^T$	$(1, 5.66, 5.66, 16, 16, 15.91)^T$	-1	2.9598
$\mathbf{x}_6$	$(6, 3)^T$	$(1, 8.49, 4.24, 36, 9, 25.46)^T$	-1	3.8502

Nonlinear SVM: Inhomogeneous Quadratic Kernel

We compute the weight vector for the hyperplane:

$$\mathbf{w} = \sum_{\alpha_i > 0} \alpha_i y_i \phi(\mathbf{x}_i) = (0, -1.413, -3.298, 0.256, 0.82, -0.018)^T$$

as well as the bias:

$$b = -8.841$$

The decision boundary in input space corresponds to an ellipse, centered at $(4.046, 2.907)$, with axis lengths 2.78 and 1.55.

Notice that we explicitly transformed all the points into the feature space just for illustration purposes.

The kernel trick allows us to achieve the same goal using only the kernel function.

SVM Training Algorithms

Instead of dealing explicitly with the bias b , we map each point $\mathbf{x}_i \in \mathbb{R}^d$ to the point $\mathbf{x}'_i \in \mathbb{R}^{d+1}$ as follows:

$$\mathbf{x}'_i = (x_{i1}, \dots, x_{id}, 1)^T$$

We also map the weight vector to $\mathbb{R}^{d+1}$, with $w_{d+1} = b$, so that

$$\mathbf{w} = (w_1, \dots, w_d, b)^T$$

The equation of the hyperplane is then given as follows:

$$h(\mathbf{x}') : \mathbf{w}^T \mathbf{x}' = w_1 x_{i1} + \dots + w_d x_{id} + b = 0$$

After the mapping, the constraint $\sum_{i=1}^n \alpha_i y_i = 0$ does not apply in the SVM dual formulations. The new set of constraints is given as

$$y_i \mathbf{w}^T \mathbf{x} \geq 1 - \xi_i$$

Dual Optimization: Gradient Ascent

The dual optimization objective for hinge loss is given as

$$\max_{\alpha} J(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$

subject to the constraints $0 \leq \alpha_i \leq C$ for all $i = 1, \dots, n$. Here $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_n)^T \in \mathbb{R}^n$.

The gradient or the rate of change in the objective function at α is given as the partial derivative of $J(\alpha)$ with respect to α , that is, with respect to each α_k :

$$\nabla J(\alpha) = \left(\frac{\partial J(\alpha)}{\partial \alpha_1}, \frac{\partial J(\alpha)}{\partial \alpha_2}, \dots, \frac{\partial J(\alpha)}{\partial \alpha_n} \right)^T$$

where the k th component of the gradient is obtained by differentiating $J(\alpha_k)$ with respect to α_k :

$$\frac{\partial J(\alpha)}{\partial \alpha_k} = \frac{\partial J(\alpha_k)}{\partial \alpha_k} = 1 - y_k \left(\sum_{i=1}^n \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}_k) \right)$$

Stochastic Gradient Ascent

Starting from an initial α , the gradient ascent approach successively updates by moving in the direction of the gradient $\nabla J(\alpha)$:

$$\alpha_{t+1} = \alpha_t + \eta_t \nabla J(\alpha_t)$$

where α_t is the estimate at the t th step, and η_t is the step size.

The optimal step size is:

$$\eta_k = \frac{1}{K(\mathbf{x}_k, \mathbf{x}_k)}$$

Instead of updating the entire α vector in each step, in the stochastic gradient ascent approach, we update each component α_k independently and immediately use the new value to update other components. The update rule for the k -th component is given as

$$\alpha_k = \alpha_k + \eta_k \frac{\partial J(\alpha)}{\partial \alpha_k} = \alpha_k + \eta_k \left(1 - y_k \sum_{i=1}^n \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}_k) \right)$$

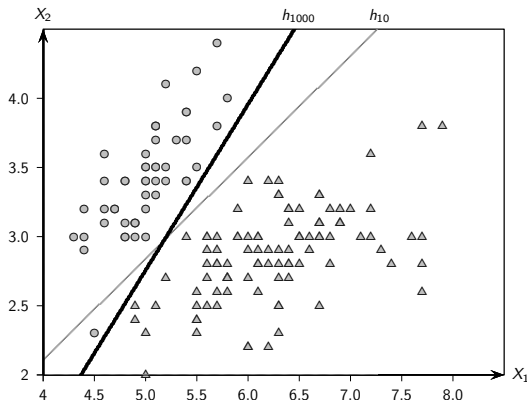
Algorithm SVM-Dual

SVM-Dual (D, K, C, ϵ):

```
1  foreach  $\mathbf{x}_i \in D$  do  $\mathbf{x}_i \leftarrow \begin{pmatrix} \mathbf{x}_i \\ 1 \end{pmatrix}$ 
2  if  $\text{loss} = \text{hinge}$  then
3     $K \leftarrow \{K(\mathbf{x}_i, \mathbf{x}_j)\}_{i,j=1,\dots,n}$  // kernel matrix, hinge loss
4  else if  $\text{loss} = \text{quadratic}$  then
5     $K \leftarrow \{K(\mathbf{x}_i, \mathbf{x}_j) + \frac{1}{2C} \delta_{ij}\}_{i,j=1,\dots,n}$  // kernel matrix, quadratic loss
6  for  $k = 1, \dots, n$  do  $\eta_k \leftarrow \frac{1}{K(\mathbf{x}_k, \mathbf{x}_k)}$ 
7   $t \leftarrow 0$ 
8   $\alpha_0 \leftarrow (0, \dots, 0)^T$ 
9  repeat
10    $\alpha \leftarrow \alpha_t$ 
11   for  $k = 1$  to  $n$  do
12     // update  $k$ th component of  $\alpha$ 
13      $\alpha_k \leftarrow \alpha_k + \eta_k \left(1 - y_k \sum_{i=1}^n \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}_k)\right)$ 
14     if  $\alpha_k < 0$  then  $\alpha_k \leftarrow 0$ 
15     if  $\alpha_k > C$  then  $\alpha_k \leftarrow C$ 
16    $\alpha_{t+1} = \alpha$ 
17    $t \leftarrow t + 1$ 
18 until  $\|\alpha_t - \alpha_{t-1}\| \leq \epsilon$ 
```

SVM Dual Algorithm: Iris Data – Linear Kernel

c_1 : Iris-setosa (circles) and c_2 : other types of Iris flowers (triangles)



Hyperplane h_{10} uses $C = 10$ and h_{1000} uses $C = 1000$:

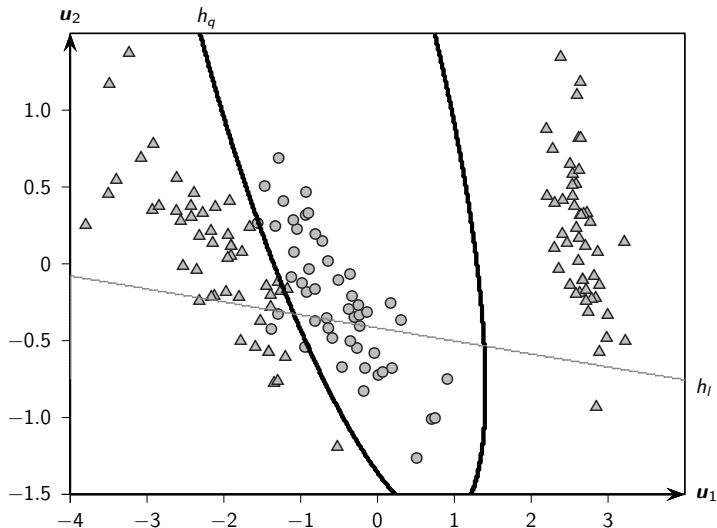
$$h_{10}(x): 2.74x_1 - 3.74x_2 - 3.09 = 0$$

$$h_{1000}(x): 8.56x_1 - 7.14x_2 - 23.12 = 0$$

h_{10} has a larger margin, but also a larger slack; h_{1000} has a smaller margin, but it minimizes the slack.

SVM Dual Algorithm: Quadratic versus Linear Kernel

c_1 : Iris-versicolor (circles) and c_2 : other types of Iris flowers (triangles)



Primal Solution: Newton Optimization

Consider the primal optimization function for soft margin SVMs. With $\mathbf{w}, \mathbf{x}_i \in \mathbb{R}^{d+1}$, we have to minimize the objective function:

$$\min_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n (\xi_i)^k$$

subject to the linear constraints:

$$y_i (\mathbf{w}^T \mathbf{x}_i) \geq 1 - \xi_i \text{ and } \xi_i \geq 0 \text{ for all } i = 1, \dots, n$$

Rearranging the above, we obtain an expression for ξ_i

$$\begin{aligned} \xi_i &\geq 1 - y_i (\mathbf{w}^T \mathbf{x}_i) \text{ and } \xi_i \geq 0, \text{ which implies that} \\ \xi_i &= \max \{0, 1 - y_i (\mathbf{w}^T \mathbf{x}_i)\} \end{aligned}$$

Primal Solution: Newton Optimization, Quadratic Loss

The objective function can be rewritten as

$$\begin{aligned} J(\mathbf{w}) &= \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \max \{0, 1 - y_i (\mathbf{w}^T \mathbf{x}_i)\}^k \\ &= \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{y_i(\mathbf{w}^T \mathbf{x}_i) < 1} (1 - y_i(\mathbf{w}^T \mathbf{x}_i))^k \end{aligned}$$

For quadratic loss, we have $k = 2$ and the gradient or the rate of change of the objective function at $\mathbf{w}$ is given as the partial derivative of $J(\mathbf{w})$ with respect to $\mathbf{w}$:

$$\nabla_{\mathbf{w}} = \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} = \mathbf{w} - 2C\mathbf{v} + 2C\mathbf{S}\mathbf{w}$$

where the vector $\mathbf{v}$ and the matrix $\mathbf{S}$ are given as

$$\begin{aligned} \mathbf{v} &= \sum_{y_i(\mathbf{w}^T \mathbf{x}_i) < 1} y_i \mathbf{x}_i & \mathbf{S} &= \sum_{y_i(\mathbf{w}^T \mathbf{x}_i) < 1} \mathbf{x}_i \mathbf{x}_i^T \end{aligned}$$

Primal Solution: Newton Optimization, Quadratic Loss

The *Hessian matrix* is defined as the matrix of second-order partial derivatives of $J(\mathbf{w})$ with respect to $\mathbf{w}$, which is given as

$$\mathbf{H}_{\mathbf{w}} = \frac{\partial \nabla_{\mathbf{w}}}{\partial \mathbf{w}} = \mathbf{I} + 2\mathbf{C}\mathbf{S}$$

Because we want to minimize the objective function $J(\mathbf{w})$, we should move in the direction opposite to the gradient. The Newton optimization update rule for $\mathbf{w}$ is given as

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \mathbf{H}_{\mathbf{w}_t}^{-1} \nabla_{\mathbf{w}_t}$$

where $\eta_t > 0$ is a scalar value denoting the step size at iteration t .

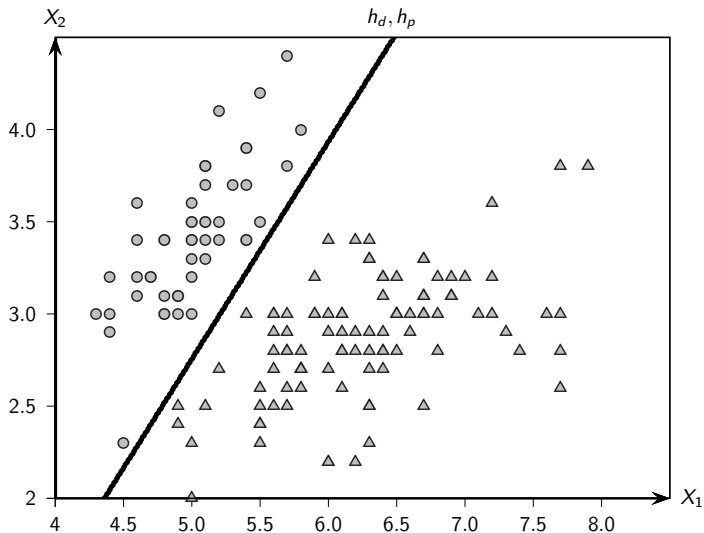
Primal SVM Algorithm

SVM-Primal (D, C, ϵ):

```
1 foreach  $x_i \in D$  do
2    $\left[ \begin{array}{l} x_i \leftarrow \begin{pmatrix} \mathbf{x}_i \\ 1 \end{pmatrix} \end{array} \right]$  // map to  $\mathbb{R}^{d+1}$ 
3  $t \leftarrow 0$ 
4  $\mathbf{w}_0 \leftarrow (0, \dots, 0)^T$  // initialize  $\mathbf{w}_t \in \mathbb{R}^{d+1}$ 
5 repeat
6    $\mathbf{v} \leftarrow \sum_{y_i(\mathbf{w}_t^T \mathbf{x}_i) < 1} y_i \mathbf{x}_i$ 
7    $\mathbf{S} \leftarrow \sum_{y_i(\mathbf{w}_t^T \mathbf{x}_i) < 1} \mathbf{x}_i \mathbf{x}_i^T$ 
8    $\nabla \leftarrow (\mathbf{I} + 2C\mathbf{S})\mathbf{w}_t - 2C\mathbf{v}$  // gradient
9
10   $\mathbf{H} \leftarrow \mathbf{I} + 2C\mathbf{S}$  // Hessian
11
12   $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t - \eta_t \mathbf{H}^{-1} \nabla$  // Newton update rule
13 until  $\|\mathbf{w}_t - \mathbf{w}_{t-1}\| \leq \epsilon$ 
```

SVMs: Dual and Primal Solutions

c_1 : Iris-setosa (circles) and c_2 : other types of Iris flowers (triangles)



SVM Primal Kernel Algorithm: Newton Optimization

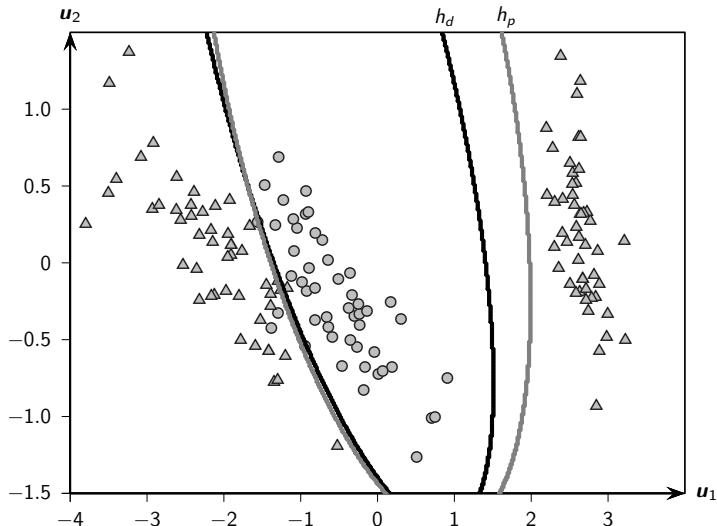
The linear soft margin primal algorithm, with quadratic loss, can easily be extended to work on any kernel matrix $\mathbf{K}$:

SVM-Primal-Kernel ($\mathbf{D}, \mathbf{K}, C, \epsilon$):

```
1 foreach  $x_i \in \mathbf{D}$  do
2    $\left[ \begin{array}{l} x_i \leftarrow \begin{pmatrix} x_i \\ 1 \end{pmatrix} // \text{ map to } \mathbb{R}^{d+1} \end{array} \right.$ 
3  $\mathbf{K} \leftarrow \{K(x_i, x_j)\}_{i,j=1,\dots,n}$  // compute kernel matrix
4  $t \leftarrow 0$ 
5  $\beta_0 \leftarrow (0, \dots, 0)^T$  // initialize  $\beta_t \in \mathbb{R}^n$ 
6 repeat
7    $\mathbf{v} \leftarrow \sum_{y_i(\mathbf{K}_i^T \beta_t) < 1} y_i \mathbf{K}_i$ 
8    $\mathbf{S} \leftarrow \sum_{y_i(\mathbf{K}_i^T \beta_t) < 1} \mathbf{K}_i \mathbf{K}_i^T$ 
9    $\nabla \leftarrow (\mathbf{K} + 2C\mathbf{S})\beta_t - 2C\mathbf{v}$  // gradient
10   $\mathbf{H} \leftarrow \mathbf{K} + 2C\mathbf{S}$  // Hessian
11   $\beta_{t+1} \leftarrow \beta_t - \eta_t \mathbf{H}^{-1} \nabla$  // Newton update rule
12   $t \leftarrow t + 1$ 
13 until  $\|\beta_t - \beta_{t-1}\| < \epsilon$ 
```

SVM Quadratic Kernel: Dual and Primal Solutions

c_1 : Iris-versicolor (circles) and c_2 : other types of Iris flowers (triangles)



Data Mining and Machine Learning

A Master Course

Master program on Data Science

Tran Trong Hieu¹

¹VNU - University of Engineering and Technology

Chapter 21: Support Vector Machines